

MARAN SUITE SYSTEM — Especificaciones de Desarrollo: Módulo Eventos

Fecha: Marzo 2026

Módulo: Eventos

Prioridad: Alta — Cierre operativo antes de migración a PostgreSQL

Contexto general

El módulo de Eventos ya tiene estructura funcional: salones, agenda semanal, CRUD de eventos, tipos de cargo y cargos por evento. Las siguientes tareas completan el módulo a nivel operativo real, con especial atención al flujo de facturación y al nuevo tipo "Evento por Mesa".

Las tareas están ordenadas de menor a mayor complejidad. Ejecutar en orden.

TAREA 1 — Confirmación antes de cancelar un evento

Prioridad: Alta | **Complejidad:** Baja | **Área:** Frontend

Descripción

Al hacer clic en "Cancelar evento" (o cambiar el estado a `cancelled`), mostrar un dialog de confirmación antes de ejecutar la acción.

Cambios requeridos

`client/src/pages/events.tsx`

Reemplazar la acción directa de cancelación por un dialog de confirmación con el texto: *"¿Estás seguro que querés cancelar el evento '[Nombre del evento]'? Esta acción marcará el evento como cancelado y no se puede deshacer."*

Botones:

- **"Sí, cancelar evento"** → color destructivo (rojo), ejecuta el PATCH con `status: "cancelled"`.

- **"Volver"** → cierra el dialog sin hacer nada.

Aplicar esta confirmación en todos los lugares donde se puede cancelar un evento: lista de eventos, detalle del evento y agenda semanal.

TAREA 2 — Validación de superposición de eventos en el mismo salón

Prioridad: Alta | **Complejidad:** Baja | **Área:** Backend

Descripción

Al crear o editar un evento, el backend debe verificar que el salón (`eventRoomId`) no tenga otro evento activo que se superponga en fecha y horario. Si hay superposición, rechazar la operación con un mensaje claro.

Lógica de validación

Un evento ocupa desde `startDate + startTime` hasta `endDate + endTime`.

Hay superposición si:

```
nuevoInicio < existenteFin AND nuevoFin > existenteInicio
```

Solo considerar eventos con estado: `inquiry`, `quoted`, `confirmed`, `in_progress`.

Excluir el propio evento al editar (por `id`).

Cambios requeridos

`server/routes.ts` — **POST `/api/events` y PATCH `/api/events/:id`**

Antes de insertar o actualizar, ejecutar la consulta de superposición.

Si hay conflicto, retornar:

```
{
  "error": "Superposición de evento",
  "message": "El salón 'Salón Principal' ya tiene el evento 'Cena de Gala' reservado"
}
```

HTTP status: `409 Conflict`

`client/src/pages/events.tsx` — **Formulario de evento**

Mostrar el mensaje de error del servidor en un toast o alerta visible.

No cerrar el modal si hay error de superposición.

TAREA 3 — Folio completo de evento: cargos, pagos anticipados y cierre con facturación

Prioridad: Alta | **Complejidad:** Alta | **Área:** Backend + Frontend

Descripción

El módulo de eventos no tiene un sistema de cierre/facturación real. Debe agregarse un folio por evento con soporte de cargos múltiples, pagos anticipados (señas), múltiples métodos de pago y emisión de comprobante. El flujo es similar al cierre de mesa del restaurante y al folio de SPA.

3.1 — Cambios en el schema (`shared/schema.ts`)

Nueva tabla `eventPayments` :

```
export const eventPayments = pgTable("event_payments", {
  id: serial("id").primaryKey(),
  eventId: integer("event_id").notNull().references(() => events.id),
  amount: numeric("amount", { precision: 10, scale: 2 }).notNull(),
  method: text("method").notNull(),
  // Métodos: "cash", "debit_card", "credit_card", "transfer", "mercadopago", "ro
  isAdvance: boolean("is_advance").default(false), // true = señal/anticipo
  reservationId: integer("reservation_id").references(() => reservations.id), //
  notes: text("notes"),
  paidAt: timestamp("paid_at").defaultNow(),
  createdAt: timestamp("created_at").defaultNow(),
});
```

Tabla `events` — agregar campos:

```
receiptType: text("receipt_type"), // "ticket", "factura_a", "factura_b", "factur
closedAt: timestamp("closed_at"),
totalAmount: numeric("total_amount", { precision: 10, scale: 2 }),
totalPaid: numeric("total_paid", { precision: 10, scale: 2 }),
```

3.2 — Nuevos endpoints (`server/routes.ts`)

POST	<code>/api/events/:id/payments</code>	→ Registrar pago (incluye anticipos)
GET	<code>/api/events/:id/payments</code>	→ Listar pagos del evento
DELETE	<code>/api/events/:id/payments/:payId</code>	→ Eliminar pago (solo si evento no está c
GET	<code>/api/events/:id/summary</code>	→ Resumen: cargos + pagos + saldo pendien
POST	<code>/api/events/:id/close</code>	→ Cerrar evento con <code>receiptType</code>

Lógica del endpoint `/close` :

- Validar que `totalPaid >= totalAmount`.
 - Si hay saldo pendiente, retornar error con el monto adeudado.
 - Si algún pago tiene `method = "room_charge"`, crear cargo en la tabla `charges` de la reserva indicada con descripción "Eventos - [nombre del evento]".
 - Cambiar estado del evento a `invoiced` y guardar `closedAt`.
-

3.3 — Cambios en el frontend (`client/src/pages/events.tsx`)

Nuevo botón "Folio / Facturar" visible en el detalle del evento cuando el estado es `confirmed`, `in_progress` o `completed`.

Modal de folio del evento con dos secciones:

Sección izquierda — Cargos:

- Lista de cargos existentes del evento (ya cargados en el módulo actual) con descripción, cantidad, precio unitario y subtotal.
- Botón "Agregar cargo" con los tipos de cargo configurables ya existentes.
- Total calculado automáticamente.

Sección derecha — Pagos:

- Lista de pagos registrados, con etiqueta "SEÑA" para los anticipos (`isAdvance: true`).
- Botón "Registrar anticipo" disponible desde cualquier estado previo al cierre.
- Al cerrar: botón "Agregar pago" con selector de método:
 - Efectivo, Débito, Crédito, Transferencia, MercadoPago, Cheque, Cargo a habitación.

- Si elige "Cargo a habitación": buscador de reservas activas por habitación o nombre del huésped.
- **Saldo pendiente** calculado en tiempo real: `total cargos - suma de pagos`.
- Botón "Cerrar y facturar" habilitado solo cuando saldo = 0.
- Dropdown de tipo de comprobante: Ticket / Factura A / Factura B / Factura C.

Anticipos desde el formulario de evento:

- En el formulario de creación/edición de evento, agregar sección "Pagos anticipados" con botón "Agregar seña".
- Campos: monto, método de pago, fecha, notas.
- Los anticipos registrados aparecen listados en el formulario y en el folio.

TAREA 4 — Tipo de evento "Evento por Mesa" con folios por mesa

Prioridad: Alta | **Complejidad:** Alta | **Área:** Backend + Frontend

Descripción

Agregar un nuevo tipo de evento llamado "Evento por Mesa" (valor: `table_event`) que habilita una gestión especial: múltiples mesas numeradas, cada una con su propio folio de cargos y pagos, facturación individual por mesa. Los cargos generales del evento (salón, equipamiento) se facturan aparte al organizador mediante el folio general del evento (Tarea 3).

Casos de uso típicos

- Cenas de Navidad o Fin de Año donde cada mesa paga su consumo.
- Eventos con venta de lugares por mesa donde cada grupo paga por separado.
- Mesas que pueden tener huéspedes del hotel (cargo a habitación).

4.1 — Cambios en el schema (`shared/schema.ts`)

Nueva tabla `eventTables` :

```
export const eventTables = pgTable("event_tables", {
  id: serial("id").primaryKey(),
  eventId: integer("event_id").notNull().references(() => events.id),
  tableNumber: integer("table_number").notNull(), // Número de mesa (ej: 1, 2, 3.
  label: text("label"), // Etiqueta opcional (ej: "Mesa VIP", "Mesa Familia Garcí
  seats: integer("seats"), // Cantidad de comensales
  status: text("status").notNull().default("open"),
  // Estados: "open", "closed", "invoiced"
  reservationId: integer("reservation_id").references(() => reservations.id), //
  createdAt: timestamp("created_at").defaultNow(),
});
```

Nueva tabla eventTableCharges :

```
export const eventTableCharges = pgTable("event_table_charges", {
  id: serial("id").primaryKey(),
  eventTableId: integer("event_table_id").notNull().references(() => eventTables.
  description: text("description").notNull(),
  quantity: numeric("quantity", { precision: 10, scale: 2 }).notNull().default("1
  unitPrice: numeric("unit_price", { precision: 10, scale: 2 }).notNull(),
  total: numeric("total", { precision: 10, scale: 2 }).notNull(),
  createdAt: timestamp("created_at").defaultNow(),
});
```

Nueva tabla eventTablePayments :

```
export const eventTablePayments = pgTable("event_table_payments", {
  id: serial("id").primaryKey(),
  eventTableId: integer("event_table_id").notNull().references(() => eventTables.
  amount: numeric("amount", { precision: 10, scale: 2 }).notNull(),
  method: text("method").notNull(),
  // Métodos: "cash", "debit_card", "credit_card", "transfer", "mercadopago", "ro
  isAdvance: boolean("is_advance").default(false),
  reservationId: integer("reservation_id").references(() => reservations.id),
  receiptType: text("receipt_type"), // al cerrar: "ticket", "factura_a", "factur
  paidAt: timestamp("paid_at").defaultNow(),
});
```

4.2 — Nuevos endpoints (server/routes.ts)

GET	/api/events/:id/tables	→ Listar mesas del evento
POST	/api/events/:id/tables	→ Agregar mesa al evento

PATCH	/api/events/:id/tables/:tableId	→ Editar mesa (label, seat)
DELETE	/api/events/:id/tables/:tableId	→ Eliminar mesa (solo si e
POST	/api/events/:id/tables/:tableId/charges	→ Agregar cargo a la mesa
DELETE	/api/events/:id/tables/:tableId/charges/:chargeId	→ Eliminar cargo
POST	/api/events/:id/tables/:tableId/payments	→ Registrar pago en mesa
GET	/api/events/:id/tables/:tableId/summary	→ Resumen: cargos + pagos
POST	/api/events/:id/tables/:tableId/close	→ Cerrar y facturar mesa i
GET	/api/events/:id/tables-summary	→ Resumen global: todas la

4.3 — Cambios en el frontend (client/src/pages/events.tsx)

En el formulario de evento:

- El campo "Tipo de evento" ya existe. Agregar el valor "Evento por Mesa" al dropdown.
- Cuando se selecciona "Evento por Mesa", mostrar un aviso: *"Este tipo de evento habilita la gestión de mesas con folios individuales."*

Nueva sección "Mesas" en el detalle del evento (visible solo cuando `type = "table_event"`):

Vista de grilla de mesas:

- Tarjetas por mesa mostrando: número de mesa, etiqueta, cantidad de comensales, estado (abierta/cerrada/facturada), saldo pendiente.
- Botón "Agregar mesa" → abre formulario con número, etiqueta opcional, comensales y opción de vincular a una reserva del hotel.
- Las mesas se pueden agregar en cualquier momento, incluso durante el evento.
- Badge de color por estado: verde (abierta), ámbar (con saldo pendiente), gris (cerrada/facturada).

Modal de folio por mesa (al hacer clic en una mesa):

Igual estructura que el folio de evento general (Tarea 3) pero para esa mesa específica:

- Sección izquierda: cargos de la mesa con botón "Agregar cargo" (descripción libre, precio, cantidad).
- Sección derecha: pagos registrados, anticipos con badge "SEÑA", saldo en tiempo

real.

- Si la mesa tiene `reservationId` asignado, mostrar opción “Cargar a habitación [N°]” preseleccionada.
- Botón “Cerrar mesa” habilitado solo con saldo = 0, con selector de comprobante.
- Una vez cerrada, la mesa queda bloqueada (no se pueden agregar más cargos ni pagos).

Vista de resumen global del evento por mesas:

- Tabla con todas las mesas: número, comensales, total cargos, total pagado, saldo, estado.
 - Total general del evento (suma de todas las mesas + cargos generales del organizador).
 - Indicador de mesas pendientes de cierre.
-

Notas para Replit

- Ejecutar las tareas **en el orden indicado** (1 → 4).
 - La Tarea 4 depende de que la Tarea 3 esté implementada (el folio general del evento debe existir antes de agregar el folio por mesa).
 - El tipo `table_event` debe agregarse al enum/validación de tipos de evento en `shared/schema.ts`.
 - El `MemStorage` (`server/storage.ts`) debe actualizarse con las nuevas tablas `eventTables` , `eventTableCharges` y `eventTablePayments` .
 - Los cargos generales del evento (salón, equipamiento) y los cargos por mesa son independientes y se facturan por separado: los generales al organizador, los de mesa a cada mesa.
 - Mantener la nomenclatura existente (camelCase en TypeScript, snake_case en SQL).
 - Los textos de interfaz deben estar en español, consistente con el resto del sistema.
-

